

Old Sessions - Solve Writeup

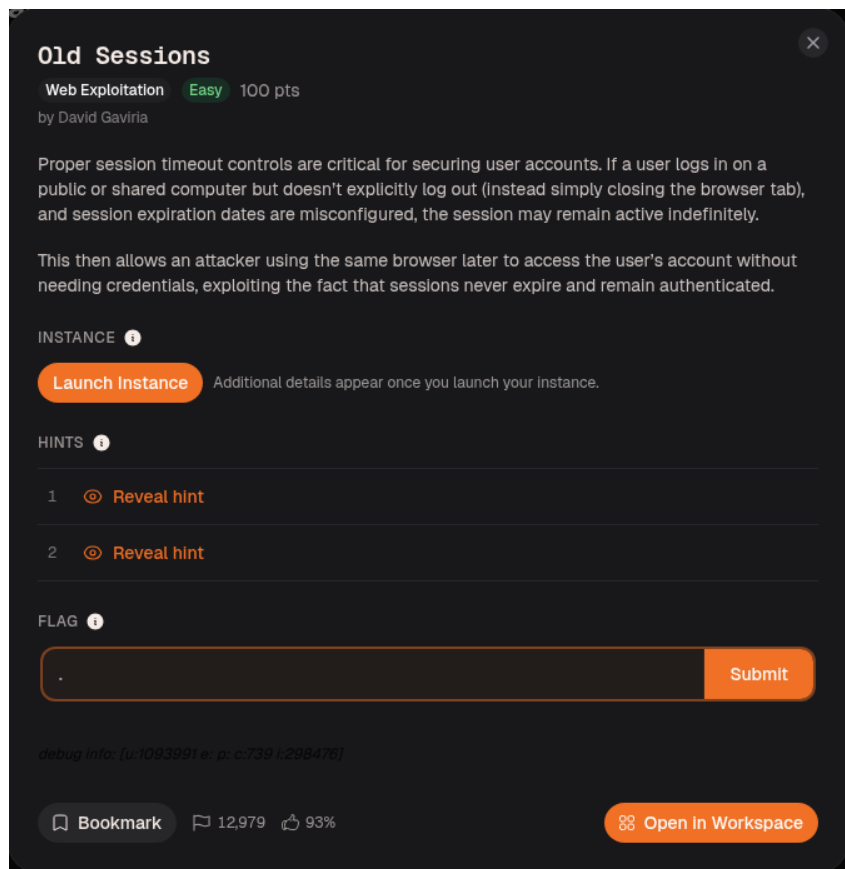
picoCTF Web Exploitation | Beginner | Broken session management

Final Flag

`picoCTF{s3t_s3ss10n_3xp1rat10n5_ed8964c2}`

Challenge Idea

The challenge description talked about old browser sessions that remain active if expiration is misconfigured. So before trying SQL injection or brute force, the main direction was to inspect how the website handles sessions and cookies.



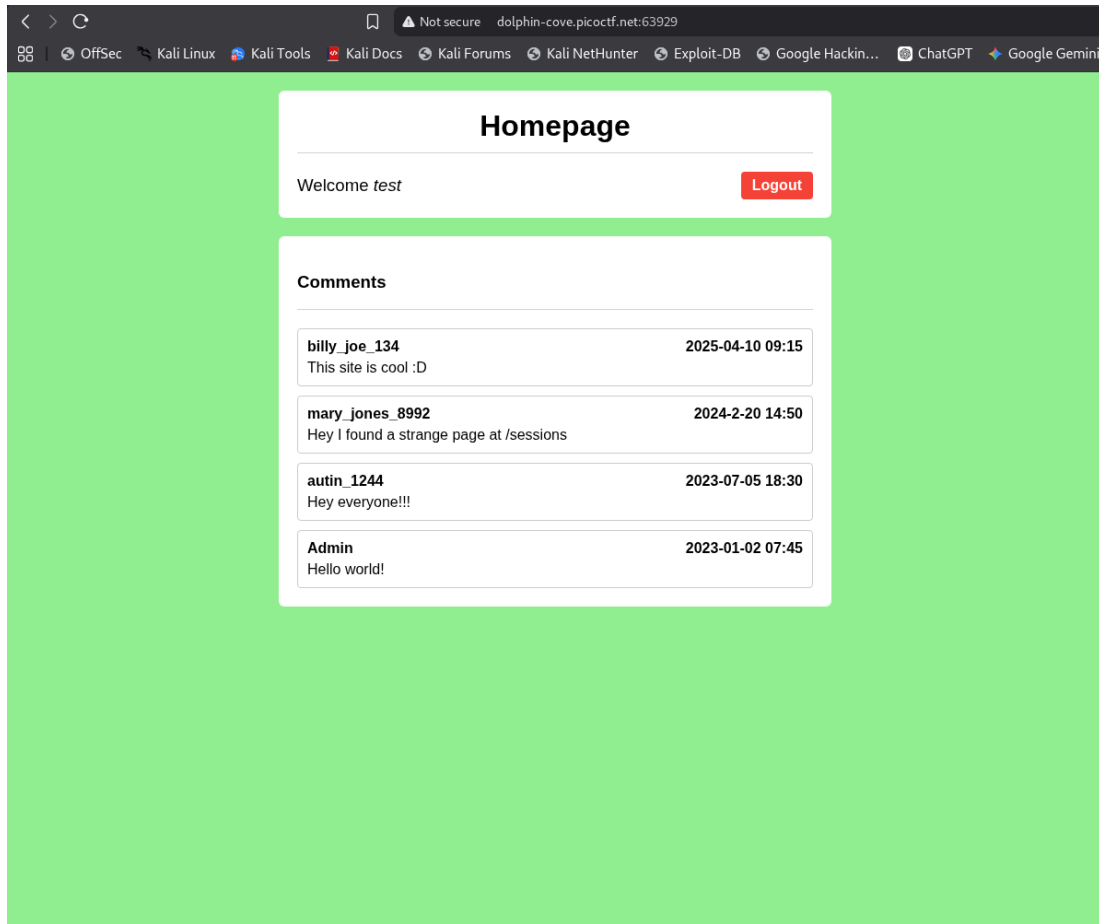
Screenshot 1 - Challenge prompt. The title and description point toward old/non-expiring sessions.

Step 1 - Register and Observe

The site started with a login/register page. I registered a normal test account and landed on the homepage. This proved account creation worked and gave me a normal user session cookie.

On the homepage, the comments were more important than the login form. One user left a clue:

mary_jones_8992: Hey I found a strange page at /sessions



Screenshot 2 - After registering, the comments revealed the hidden route /sessions.

Step 2 - Visit /sessions

I visited the hinted path /sessions. This page leaked session records. It showed my test user's session and also an admin session. That is a serious session management bug because a session token is enough to impersonate the user who owns it.

Example leak from /sessions:

- 1) session:hwfoMncI6_IXEk0BnLH4JFq9VILf4fpmQfDGGSuM5Rk, {'_permanent': True, 'key': 'admin'}
- 2) session:GUpaLh4g7TsKHgQcxQMiiAmvv4vE0dYDFUesHp5BPUk, {'_permanent': True, 'key': 'test'}

The important leaked value was the admin session. My own account was 'test', so the other session belonged to 'admin'.

Step 3 - Hijack the Admin Session

In the browser Developer Tools, I opened Storage/Application - Cookies and replaced my current session cookie value with the leaked admin session token. After refreshing the homepage, the server trusted that cookie and treated me as admin.

Browser path:
Inspect -> Application/Storage -> Cookies -> session

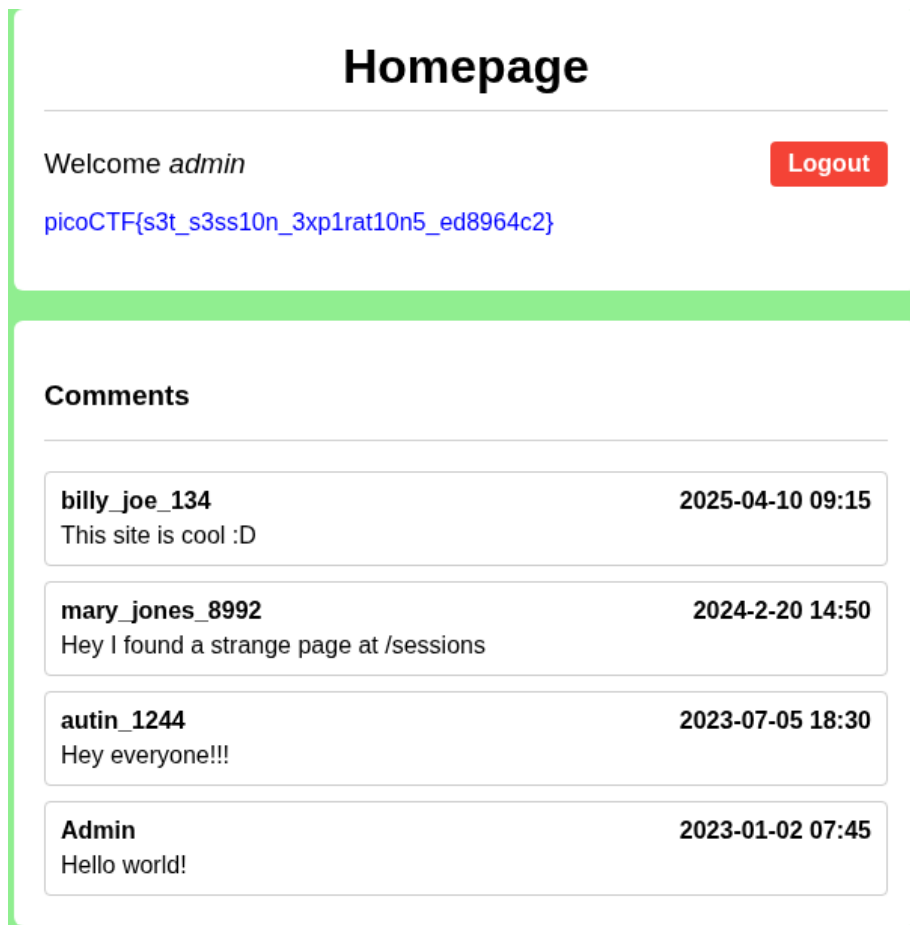
Replace my session value with the leaked admin session value.
Then refresh the homepage.

The same idea could also be tested with curl:

```
curl -i -b "session=hwfoMncI6_IXEk0BnLH4JFq9VILF4fpmQfDGGSuM5Rk" http://dolphin-cove.picoctf.net:63929/
```

Step 4 - Flag Appears

After the cookie swap, the page changed from 'Welcome test' to 'Welcome admin' and displayed the flag.



Screenshot 3 - The leaked admin session gave admin access and revealed the flag.

Why This Worked

- The /sessions endpoint exposed session tokens.
- The admin session token was still valid.
- The server identified users only by the session cookie.
- Replacing the cookie value with the admin session value caused session hijacking.

Security Lesson

This was a broken session management issue. Session IDs must be secret, must expire correctly, and should be invalidated on logout. An exposed valid session cookie is equivalent to exposing the user's password for that active session.

Short Solve Path

1. Register a normal user.
2. Read homepage comments.
3. Notice the clue: /sessions.
4. Visit /sessions and find the admin session token.
5. Replace my session cookie with the admin session token.
6. Refresh homepage.
7. Read the flag.

Flag: picoCTF{s3t_s3ss10n_3xp1rat10n5_ed8964c2}